智能物联网系统设计实验报告

——基于 ESP32 与华为云的智能图书馆系统

姓名 陈文浩

学号 3240101166

院系 信息与电子工程学院

2026 年 4 月 28 日

摘　要

本实验设计并实现了一套基于 ESP32 与华为云的智能图书馆物联网系统。系统以 ESP32 DevKit V1 为核心控制器，集成 RC522 NFC 读卡器、条码扫描模块、BMP280 温度传感器、DHT11 湿度传感器、光敏电阻、OLED 显示屏、RGB 指示灯和风扇执行机构，实现图书注册、借还状态感知、环境数据采集、湿度联动除湿和本地状态显示等功能。云端侧主要使用华为云 IoTDA 完成设备接入、属性上报、设备影子查询和命令下发，使用 OBS 对象存储保存图书档案、借还历史和阅读笔记。需要主动访问云端接口的操作大多集成在微信小程序端完成，包括设备影子读取、图书信息查询、OBS 文件读写和远程控制；FunctionGraph 仅作为 ISBN 查询与命令下发的一种可选尝试方案，并非系统所有云端功能的实现核心。测试结果表明，系统能够稳定完成设备、小程序与云平台之间的数据同步，借还事件响应及时，环境监测与风扇控制逻辑有效，具备较完整的智能图书馆原型系统价值。

目录

1 项目成员与分工 **5**

2 引言 **5**

2.1 项目背景 5

2.2 设计目标 5

3 系统总体设计 **6**

3.1 总体架构 6

3.2 系统工作原理 6

3.3 数据流设计 6

4 硬件系统设计 **7**

4.1 硬件清单 7

4.2 硬件构成原理图 7

4.3 引脚分配 7

4.4 图书识别模块 9

4.5 环境监测与风扇控制 9

5 ESP32 固件设计 **9**

5.1 开发环境与依赖 9

5.2 状态机设计 10

5.3 图书借还逻辑 10

5.4 MQTT 属性上报与命令处理 11

6 云端服务设计 **12**

6.1 IoTDA 设备接入 12

6.2 FunctionGraph 补充探索 12

6.3 OBS 数据持久化 12

7 微信小程序设计 **13**

7.1 页面结构 13

7.2 主页面功能 13

7.3 书架与笔记功能 15

7.4 AI 对话功能 15

8 软件源代码组织 **15**

9 实验过程 **15**

10 系统调试与问题解决 **17**

11 实验结果 **18**

11.1 功能测试 18

11.2 性能与稳定性 18

12 总结与展望 **19**

12.1 项目成果 19

12.2 不足与改进方向 19

1 项目成员与分工

本项目由一名成员完成，主要工作包括需求分析、硬件选型与接线、ESP32 固件开发、华为云 IoTDA/OBS 配置、微信小程序开发、AI 对话功能集成、系统调试与实验报告撰写。FunctionGraph 部分作为补充探索进行实现和验证，但最终主要业务闭环由小程序端承担。

- **陈文浩**：负责项目全部设计与实现工作，贡献比例为 100%。

2 引言

2.1 项目背景

传统小型图书管理通常依赖人工登记或单次扫码，无法自然记录图书取放状态，也缺少环境监测、远程查看和阅读过程数据沉淀。随着物联网、云平台和移动端应用的发展，可以将图书标签识别、传感器采集、云端同步和移动端交互组合起来，构建低成本、可扩展的智能图书馆系统。

本项目选择“智能图书馆”作为综合实验对象，原因在于该场景同时包含感知、通信、控制、云端服务和用户交互：图书本身需要身份识别，书架环境需要实时监测，借还状态需要跨端同步，用户还希望通过手机查看书架状态并记录阅读想法。因此，该项目能够较完整地体现智能物联网系统设计的主要环节。

2.2 设计目标

系统设计目标如下：

1. 使用 NFC 标签和条码扫描实现图书注册与身份绑定；
2. 通过 ESP32 采集温度、湿度、光照等环境信息，并上报至华为云 IoTDA；
3. 检测图书借出和归还事件，维护本地图书状态与云端状态；
4. 通过微信小程序展示环境数据、藏书统计、借还历史和设备状态；
5. 支持远程下发 RGB 指示灯、风扇控制和图书信息更新命令；
6. 使用 OBS 保存图书 JSON 档案、阅读时长、借还历史和读书笔记；
7. 集成 AI 对话能力，根据书架知识库回答图书和设备相关问题。

3 系统总体设计

3.1 总体架构

系统采用典型的物联网三层架构：感知层、网络/平台层和应用层。

- **感知层**：由 ESP32、RC522、条码扫描模块、BMP280、DHT11、光敏电阻、OLED、RGB LED 和风扇组成，负责图书身份识别、环境采集、本地显示和执行控制。
- **网络与平台层**：ESP32 通过 WiFi 接入互联网，使用 MQTT 与华为云 IoTDA 通信；IoTDA 保存设备影子并提供命令下发能力；OBS 用于持久化图书数据。
- **应用层**：微信小程序作为用户入口，负责设备影子查询、命令下发、OBS 图书数据读写、书架展示和 AI 问答。

3.2 系统工作原理

系统的基本工作原理可以概括为“本地感知、边缘判断、云端同步、移动端交互”。ESP32 作为边缘节点直接连接各类传感器和执行器：一方面周期性读取温度、湿度和光照数据，另一方面监听 NFC 与条码扫描事件。对于普通环境数据，ESP32 直接通过 MQTT 上报至 IoTDA；对于图书事件，ESP32 先在本地完成 UID 查询、状态翻转和阅读时长计算，再将关键状态同步到云端。微信小程序周期性读取设备影子，并在检测到新书注册、借出、归还等状态变化时，进一步调用图书信息 API 和 OBS 接口完成图书档案持久化。

这种设计使设备端即使在短时间网络波动时也可以保留基本状态，同时把复杂的云端 API 调用和界面交互放在小程序端处理，降低了 ESP32 固件的复杂度。IoTDA 在系统中承担设备状态中转和命令通道角色，OBS 承担轻量级数据存储角色，小程序承担业务编排和用户交互角色。

3.3 数据流设计

系统核心数据流分为四类：

1. **环境数据流**：ESP32 定时读取温度、湿度和光照值，通过 MQTT 属性上报写入 IoTDA 设备影子，小程序每 5 秒查询设备影子并刷新界面。
2. **新书注册流**：用户先将 NFC 标签贴在书上，ESP32 检测到未知 UID 后进入等待扫码状态；条码扫描模块读取 ISBN 后，ESP32 将 UID 与 ISBN 绑定并上报注册事件。

3. **图书信息补全流：**小程序检测到新书 ISBN 后调用 `data.isbn.work` 接口查询书名、作者、出版社等信息，并将结果写入 OBS，同时可通过 IoTDA 命令下发给 ESP32。FunctionGraph 也实现了类似流程，但在最终演示中不是主路径。

4. **借还记录流：**已注册图书再次被 NFC 检测时，ESP32 将其状态在“在架”和“借出”之间切换，并上报图书 UID、ISBN、状态、借出次数和阅读时长；小程序监听状态变化后更新 OBS 中对应图书的 JSON 文件。

4 硬件系统设计

4.1 硬件清单

模块	型号/说明	数量
主控开发板	ESP32 DevKit V1	1
NFC 读卡模块	RC522，13.56MHz	1
条码扫描模块	M803D，UART 通信	1
温度传感器	BMP280，I2C 通信	1
湿度传感器	DHT11，单总线通信	1
光照检测	光敏电阻分压输入 ADC	1
显示模块	0.96 寸 I2C OLED，SSD1306	1
指示灯	RGB LED	1
执行机构	5V 直流风扇	1
驱动器件	PNP 三极管及保护电路	1

表 1: 主要硬件模块清单

4.2 硬件构成原理图

项目硬件围绕 ESP32 主控展开，输入侧包括图书识别与环境感知模块，输出侧包括显示、指示和风扇执行模块。各模块通过 SPI、UART、I2C、ADC、GPIO/PWM 等接口与 ESP32 相连，硬件构成原理如图 1 所示。

4.3 引脚分配

根据最终固件 `src/SmartLibrary/SmartLibrary.ino`，系统主要引脚分配如表 2 所示。

输入与感知模块	**ESP32 DevKit V1 主控**	输出与执行模块
RC522 NFC 读卡器 HSPI：SDA/SCK/- MOSI/MISO/RST	WiFi 接入 MQTT 通信 本地状态机 SPIFFS 缓存	OLED 显示屏 I2C：SDA/SCL
条码扫描模块 UART2：TX/RX	传感器采样 图书 UID/ISBN 绑定 借还状态判断	RGB LED PWM/GPIO 输出
BMP280 温度传感器 I2C：SDA/SCL	设备影子上报 命令接收与响应	5V 风扇 PNP 三极管驱动
DHT11 湿度传感器 单总线 GPIO23	$\Longleftrightarrow$ 华为云 IoTDA / OBS / 微信小程序	蜂鸣/语音提示预留 GPIO 触发
光敏电阻 ADC GPIO32		

图 1: 项目硬件构成原理图

功能模块	ESP32 引脚	说明
RC522 NFC	SDA=15, SCK=14, MOSI=13, MISO=12, RST=4	使用 HSPI 总线
条码扫描模块	TX=33, RX=25	UART2，9600bps
I2C 总线	SDA=21, SCL=22	连接 BMP280 与 OLED
RGB LED	R=16, G=18, B=17	PWM 输出控制颜色
光敏电阻	GPIO32	ADC 输入
DHT11	GPIO23	湿度采集
风扇	GPIO19	PNP 驱动，低电平导通

表 2: ESP32 引脚分配

4.4 图书识别模块

图书身份由 NFC UID 与 ISBN 共同确定。NFC 标签用于日常取放识别，用户不需要每次借还书都扫描条码；ISBN 条码用于首次注册时查询书籍元数据。系统将二者绑定后形成完整图书档案：

- NFC UID 负责快速、无接触地识别某一本实体书；
- ISBN 负责查询书名、作者、出版社等标准图书信息；
- 本地 SPIFFS 和云端 OBS 均保存 UID 与 ISBN 的映射关系。

4.5 环境监测与风扇控制

系统使用 BMP280 读取温度，使用 DHT11 读取湿度，使用光敏电阻读取光照 ADC 值。湿度超过阈值时，风扇自动启动；用户也可以在小程序中手动开启风扇。由于风扇工作电流大于 ESP32 GPIO 可直接驱动能力，因此使用 PNP 三极管作为开关器件。最终程序中将 GPIO 低电平定义为风扇开启，高电平定义为风扇关闭，并通过 `fanManualOverride` 标志避免手动模式被自动湿度逻辑覆盖。

5 ESP32 固件设计

5.1 开发环境与依赖

固件使用 Arduino 框架开发，主要依赖如下：

- `WiFi.h` 与 `PubSubClient.h`：实现 WiFi 连接和 MQTT 通信；
- `ArduinoJson.h`：解析云端命令和构造属性上报 JSON；
- `MFRC522v2.h`：驱动 RC522 NFC 读卡模块；
- `HardwareSerial.h`：读取条码扫描模块输出；
- `SPIFFS.h`：保存本地图书缓存；
- `Adafruit_BMP280.h`、`DHTesp.h`：采集温湿度；
- `Adafruit_SSD1306.h`：驱动 OLED 显示屏。

5.2 状态机设计

固件通过 SystemState 枚举维护系统状态：

```
enum class SystemState {
  IDLE,
  AWAITING_SCAN,
  REGISTERING,
  MONITORING
};

SystemState systemState = SystemState::IDLE;
```

- IDLE：系统正常待机，等待 NFC 事件；
- AWAITING_SCAN：检测到未知 NFC 标签，等待用户扫描 ISBN；
- REGISTERING：预留注册处理中状态；
- MONITORING：监控已注册图书的借还状态。

当检测到未知 UID 时，系统进入 AWAITING_SCAN，并启动 30 秒超时计时；若在超时前扫描到 ISBN，则完成注册并上报云端；若超时，则清空待注册 UID 并返回待机状态。

5.3 图书借还逻辑

由于 RC522 感应距离较短，系统采用“再次检测同一标签即翻转状态”的方式判断借还。对已注册图书：

1. 当前为在架状态时，再次检测判定为借出，借出次数加一，记录借出时间；
2. 当前为借出状态时，再次检测判定为归还，根据借出时间计算本次阅读时长，并累计到图书记录；
3. 每次状态变化后保存本地缓存，并通过 MQTT 上报图书状态。

固件中的核心处理逻辑如下。程序先按 UID 查找图书记录，若该 UID 已存在，则翻转在架状态；若不存在，则进入等待扫码注册流程。

```
void onBookDetected(const String& uid) {
  int idx = findBookByUID(uid);
  if (idx >= 0) {
    bool wasOnShelf = bookShelf[idx].onShelf;
```

```
    bookShelf[idx].onShelf = !wasOnShelf;
    saveBooks();

    if (wasOnShelf) {
      bookShelf[idx].lastBorrowTime = millis();
      bookShelf[idx].borrowCount++;
      reportBookStatus(uid, false);
    } else {
      unsigned long readingSec =
        (millis() - bookShelf[idx].lastBorrowTime) / 1000;
      bookShelf[idx].totalReadingSec += readingSec;
      bookShelf[idx].lastBorrowTime = 0;
      reportBookStatus(uid, true);
    }
  } else {
    pendingNFCUID = uid;
    awaitingStart = millis();
    systemState = SystemState::AWAITING_SCAN;
  }
}
```

5.4 MQTT 属性上报与命令处理

ESP32 使用华为云 IoTDA 标准 MQTT 主题进行通信。属性上报主题为：

```
$oc/devices/{device_id}/sys/properties/report
```

上报内容包括温度、湿度、光照、RGB 状态、藏书总数、在架数量、借出数量和图书事件。设备订阅命令主题后可处理三类命令：

- `LEDControl`：控制 RGB 指示灯颜色和开关；
- `FanControl`：控制风扇手动开启或交还自动控制；
- `deliver_book_info`：接收云端查询到的书名、作者、出版社和出版年份。

属性上报时，固件将传感器数据和统计数据拼接为华为云 IoTDA 要求的服务属性格式，并发布到标准主题：

```
sprintf(props,
  "\"Temperature\":%.1f,\"Humidity\":%.0f,"
  "\"Photores\":%d,\"RED\":%d,\"GREEN\":%d,"
```

```
  "\"BLUE\":%d,\"Switch\":%d,"
  "\"BookCount\":%d,\"OnShelfCount\":%d,"
  "\"BorrowedCount\":%d,\"Time\":%ld}}]}",
  temperature, humidity, photoValue,
  rVal, gVal, bVal, deviceSwitch ? 1 : 0,
  bookCount, onShelfCount, borrowedCount, runningSec);

sprintf(msg,
  "{\"services\":[{\"service_id\":\"Env\",\"properties\":{%s",
  props);

mqtt.publish(TOPIC_PROP_REPORT, msg);
```

6 云端服务设计

6.1 IoTDA 设备接入

华为云 IoTDA 负责设备注册、MQTT 接入、设备影子存储和命令下发。物模型服务 ID 设置为 `Env`，主要属性包括 `Temperature`、`Humidity`、`Photores`、`BookCount`、`OnShelfCount`、`BorrowedCount`、`book_uid`、`book_isbn` 和 `book_status` 等。实际使用时，小程序通过 IoTDA REST API 查询设备影子，并从 `shadow[0].reported.properties` 路径中解析最新属性；风扇、RGB 和图书信息下发也由小程序调用 IoTDA 命令接口完成。

6.2 FunctionGraph 补充探索

项目中曾实现 FunctionGraph 函数，用于在设备上报新书注册事件后提取 ISBN、调用 `data.isbn.work` 开放接口查询图书信息，并通过 IoTDA 命令 `deliver_book_info` 将查询结果下发给设备。该方案验证了云函数参与业务处理的可行性，但最终系统没有把全部云端操作都放在 FunctionGraph 中。考虑到课程项目调试便利性和小程序界面的交互需求，图书查询、OBS 读写、设备影子读取和远程控制等主要功能均集成在小程序端完成，FunctionGraph 更适合作为后续将业务逻辑从前端迁移到云端时的基础。

6.3 OBS 数据持久化

OBS 用于保存图书长期数据，每本书对应一个 JSON 文件，路径格式为：

```
book/{isbn}.json
```

典型数据结构如下：

```
{
  "uid": "6061C645",
  "isbn": "9787540481124",
  "title": "书名",
  "author": "作者",
  "publisher": "出版社",
  "currentStatus": "on_shelf",
  "borrowCount": 1,
  "totalReadingSec": 68,
  "lastBorrowTime": 0,
  "history": [
    {"type": "borrowed", "timestamp": 1777124595384},
    {"type": "returned", "timestamp": 1777124663147, "durationSec": 68}
  ],
  "notes": [
    {"content": "阅读笔记内容", "timestamp": 1777200000000}
  ]
}
```

该设计避免了额外数据库部署，适合课程项目和轻量级个人书架场景。

7 微信小程序设计

7.1 页面结构

小程序采用三页面结构：

- **pages/Huawei_IOT**：主页面，展示环境数据、设备连接状态、藏书统计、借还动态、风扇控制、RGB 控制和调试日志；
- **pages/bookshelf/bookshelf**：书架页面，从 OBS 读取全部图书 JSON 文件，展示在架/借出状态、借出次数、阅读时长和历史记录，并支持添加阅读笔记；
- **pages/ai_chat/ai_chat**：AI 对话页面，从 OBS 汇总书架知识库，调用 DeepSeek Chat API 回答用户问题。

7.2 主页面功能

主页面首先通过 IAM API 获取 Token，然后调用 IoTDA 设备影子接口读取设备数据。页面每 5 秒自动刷新一次，并将温度、湿度、光照、图书 UID、ISBN、借还状态和

统计数量映射到界面。用户可以在主页面手动刷新数据、下发 RGB 命令、手动控制风扇，以及进入书架和 AI 页面。

小程序解析设备影子时，重点是从 IoTDA 返回结构中找到 reported properties。该逻辑解决了早期直接读取 `reported` 导致属性为空的问题。

```
parseShadowData: function(data) {
  var shadow = data.shadow ? data.shadow[0] : data;
  var reported = shadow.reported || shadow;
  var props = reported.properties || reported;

  this.setData({
    temperature: props.Temperature !== undefined
      ? props.Temperature.toFixed(1) + '°' : '--',
    photores: props.Photores || '--',
    humidity: props.Humidity || 0,
    bookUid: props.book_uid || props.BookUid || '',
    bookIsbn: props.book_isbn || props.BookIsbn || '',
    bookStatus: props.book_status || props.BookStatus || '',
    bookCount: props.BookCount || 0,
    onShelfCount: props.OnShelfCount || 0,
    borrowedCount: props.BorrowedCount || 0
  });
}
```

远程控制同样由小程序端调用 IoTDA 命令接口完成。例如风扇控制命令的核心请求如下：

```
wx.request({
  url: 'https://' + iotdahttps + '/v5/iot/' +
       projectId + '/devices/' + deviceId + '/commands',
  method: 'POST',
  header: {
    'content-type': 'application/json',
    'X-Auth-Token': token
  },
  data: JSON.stringify({
    service_id: 'Env',
    command_name: 'FanControl',
    paras: { fan: on ? 'on' : 'off' }
  })
});
```

7.3 书架与笔记功能

书架页面通过 OBS 的 ListObjects 接口列举 `book/` 前缀下的文件。由于 OBS 列举接口返回 XML，小程序使用正则表达式提取 `Key` 和 `LastModified` 字段，再逐个读取 JSON 文件并生成书架列表。点击图书卡片可查看详情、借还历史和阅读笔记。新增笔记后，小程序将笔记写入对应 JSON 文件的 `notes` 数组，实现跨端持久化。

```
var regex = /<Contents>([\s\S]*?)<\/Contents>/g;
while ((match = regex.exec(res.data)) !== null) {
  var content = match[1];
  var keyMatch =
    content.match(/<Key>([\s\S]*?)<\/Key>/);
  if (keyMatch && !keyMatch[1].endsWith('/')) {
    files.push({ key: keyMatch[1] });
  }
}
```

7.4 AI 对话功能

AI 页面启动时从 OBS 读取最多 20 本图书信息，构造包含书名、作者、在架状态和借出次数的书架知识库，再作为系统提示词的一部分发送给 DeepSeek Chat API。用户可以询问当前有哪些书、哪些书已借出、适合读哪本书、设备如何工作等问题。该功能使小程序从单纯监控界面扩展为具有一定知识服务能力的智能助手。

8 软件源代码组织

本项目软件部分经过整理后，当前主要由 ESP32 固件、微信小程序应用目录、云函数探索代码和数据样例组成。报告正文只展示关键代码片段，完整源代码随项目目录提交。主要文件结构和功能说明如表 3 所示。

从软件分层看，ESP32 固件侧只保留必要的设备控制和状态判断逻辑；小程序代码统一收纳在 `src/Application` 下，负责与用户交互和多数云端 REST API 调用；OBS 中的数据文件采用 JSON 格式，便于调试、直接查看和跨页面复用。这种组织方式使固件、应用和云端探索代码的边界更加清晰，也符合课程项目快速迭代的特点。

9 实验过程

本实验按照“单模块验证、设备端集成、云端接入、小程序联调、整体测试”的顺序推进。

文件路径	功能说明
`src/SmartLibrary/SmartLibrary.ino`	ESP32 主程序，包含 WiFi/MQTT 连接、NFC 检测、条码扫描、图书注册、借还判断、SPIFFS 本地缓存、传感器采集、OLED 显示、RGB 指示、风扇控制和串口 AI 测试逻辑。
`src/Application/app.json`	微信小程序页面注册与全局窗口配置，当前注册主页面、书架页面和 AI 对话页面。
`src/Application/pages/Huawei_IOT.js`	微信小程序主页面逻辑，负责 IAM Token 获取、设备影子读取、IoTDA 命令下发、图书信息查询、借还事件去重和 OBS 基础读写。
`src/Application/pages/bookshelf/bookshelf.js`	书架页面逻辑，负责列举 OBS 图书文件、展示图书状态、查看借还历史、统计阅读时长和保存阅读笔记。
`src/Application/pages/ai_chat/ai_chat.js`	AI 对话页面逻辑，负责从 OBS 构造书架知识库，并调用 DeepSeek API 进行问答。
`src/Application/utils/util.js`	小程序公共工具文件，保留时间格式化等辅助函数。
`cloud/functiongraph/book_lookup.py`	FunctionGraph 探索代码，实现 ISBN 查询和 IoTDA 命令下发的云函数版本，作为后续云端化扩展参考。
`data/9787540481124.json`	OBS 图书 JSON 数据格式样例，便于说明图书档案、借还历史和阅读笔记的数据结构。

表 3: 软件源代码文件说明

1. **需求分析与方案确定：**首先确定智能图书馆的核心功能，包括图书身份识别、借还状态记录、环境监测、远程控制和阅读数据管理。随后确定采用 NFC 识别日常取放、条码扫描绑定 ISBN、小程序作为主要用户界面的方案。

2. **硬件模块独立测试：**分别测试 RC522 NFC 读卡器、M803D 条码扫描模块、BMP280、DHT11、光敏电阻、OLED 和风扇驱动电路，确认各模块接线、通信接口和基本读写功能正常。

3. **ESP32 固件集成：**在独立测试基础上整合状态机、图书缓存、借还判断、传感器采集、MQTT 上报和命令处理逻辑，使设备端能够独立完成图书事件识别和环境数据采集。

4. **华为云 IoTDA 接入：**创建产品和设备，配置物模型属性与命令，完成 ESP32 MQTT 接入、属性上报和命令订阅。通过设备影子验证温度、湿度、光照、藏书统计等数据是否正确更新。

5. **微信小程序开发：**开发主页面、书架页面和 AI 对话页面。主页面负责设备影子查询和命令下发，书架页面负责 OBS 图书数据展示与笔记保存，AI 页面负责基于书架知识库进行问答。

6. **云端数据持久化：**将每本书的数据保存为 OBS 中的 JSON 文件，逐步补充注册、借出、归还、阅读时长和笔记字段，并通过小程序端完成读取、更新和展示。

7. **整体联调与演示：**按照“新书贴 NFC 标签、扫描 ISBN、查询图书信息、借出、归还、查看书架、添加笔记、AI 问答、远程控制风扇”的流程进行完整测试，确认端、云、应用三部分能够形成闭环。

10 系统调试与问题解决

项目调试过程中遇到的主要问题和解决方案如下：

1. **DHT11 读取稳定性问题**：早期驱动方案在 ESP32 上出现看门狗超时，后改用 `DHTesp` 库，提高了读取稳定性。
2. **设备影子解析路径错误**:IoTDA 返回数据嵌套在 `shadow[0].reported.properties` 中，修正小程序解析逻辑后可正确读取属性。
3. **OBS 列举返回 XML**：小程序原本按 JSON 处理失败，后改为解析 XML 中的 `Contents` 和 `Key` 字段。
4. **风扇控制电平反相**：PNP 三极管低电平导通、高电平截止，修正 GPIO 输出逻辑后风扇控制恢复正常。
5. **手动风扇控制被自动逻辑覆盖**：加入 `fanManualOverride` 标志，手动开启时跳过湿度自动控制。
6. **借还事件重复写入**：小程序使用 `lastProcessedEvent` 记录最近处理的 UID 和状态组合，减少重复触发导致的 OBS 多次写入。
7. **NFC 感应距离较短**：通过调整操作方式和采用状态翻转逻辑完成演示，后续可改进天线或更换识别方案。

11 实验结果

11.1 功能测试

11.2 性能与稳定性

根据实际运行和调试观察，系统表现如下：

- ESP32 每 10 秒上报一次环境和统计数据，小程序每 5 秒刷新一次设备影子；
- NFC 标签识别和状态切换响应较快，能够满足手动借还演示需求；
- 风扇命令从小程序下发到设备响应延迟较低，适合远程控制；
- OBS JSON 方式结构简单，便于调试和展示，但并发写入和权限控制仍有改进空间；
- AI 对话依赖外部 API 网络状态，适合作为增强功能，不应作为核心控制链路。

测试项目	预期结果	结果
WiFi 与 MQTT 连接	ESP32 成功连接 IoTDA 并订阅命令主题	通过
环境数据采集	温度、湿度、光照可定时读取并显示	通过
属性上报	设备影子显示最新环境和图书统计数据	通过
NFC 图书识别	可读取标签 UID 并触发注册或借还逻辑	通过
条码注册	可读取 ISBN 并与 NFC UID 绑定	通过
图书信息查询	可根据 ISBN 获取书名、作者、出版社	通过
OBS 持久化	可保存图书档案、历史和笔记	通过
小程序监控	可查看环境数据、藏书统计和借还状态	通过
风扇控制	支持湿度自动控制和小程序手动控制	通过
AI 对话	可基于书架知识库回答相关问题	通过

表 4: 系统功能测试结果

12 总结与展望

12.1 项目成果

本项目完成了一套从硬件感知到云端平台再到移动端应用的完整智能物联网系统。系统能够通过 NFC 与 ISBN 结合完成图书注册，通过传感器采集书架环境，通过 IoTDA 实现设备状态同步和远程命令控制，通过 OBS 保存图书档案与阅读记录，并通过微信小程序和 AI 对话提供较完整的人机交互体验。相比单一的传感器采集实验，本项目更强调端云协同、数据闭环和实际应用场景。

12.2 不足与改进方向

后续可从以下方面继续优化：

1. **识别距离**：RC522 感应距离较短，可尝试更合适的天线、NFC 标签布局或其他 RFID 方案；
2. **安全性**：当前 OBS 读写方案偏向演示，正式系统应使用签名 URL、云函数代理或更细粒度的权限控制；
3. **数据一致性**：可引入数据库或云函数统一处理借还事件，减少小程序端直接写 OBS 带来的并发风险；
4. **可视化**：可增加温湿度历史曲线、借阅趋势图和阅读时长统计；
5. **多设备扩展**：未来可支持多个书架节点，并在云端按设备 ID 汇总不同区域的藏书状态。

后续插图建议：实物接线图、ESP32 运行图、IoTDA 设备影子截图、OBS 文件列表截图、微信小程序主页面截图、书架页面截图、AI 对话页面截图。